MODULE 01 · DOMAIN 1 - INFRASTRUCTURE AS CODE WITH TERRAFORM

Infrastructure as Code, and Why Terraform

What IaC actually gives you, what it does not, and where Terraform sits among the alternatives.

DOMAIN 1 of 8 COURSE WEIGHT 6% MODULE 1 of 27

admin@sthithapragna.com

Infrastructure as Code does not make infrastructure better. It makes infrastructure **reviewable, repeatable and diffable**.

What Infrastructure as Code is

- **Machine-readable definition files** that describe infrastructure, replacing manual console clicks and one-off scripts.
- **Stored in version control** alongside application code. This is the defining practice, not an optional extra.
- The file is the **source of truth**. A tool reads it and makes reality match.
- HashiCorp's framing: define infrastructure in a **human-readable** configuration, then provision it programmatically.
- It is **not** a deployment pipeline, not a monitoring system, and not a cost tool.

> ON THE EXAM
>
> If a question asks the primary purpose of IaC, the answer is programmatic creation and configuration of resources. Nothing about pipelines or price.

Declarative or imperative?

DECLARATIVE

You describe the destination

- State what should exist, not how
- The tool computes the steps
- Re-running converges to the same result
- Terraform, CloudFormation, ARM

IMPERATIVE

You describe the journey

- State the steps, in order
- You own the sequencing
- Re-running may repeat the work
- Shell scripts, raw API calls

WHERE TERRAFORM SITS

Terraform is declarative. You never write the order of operations; the dependency graph derives it.

The advantages the exam actually rewards

- **Versioning.** Every change has an author, a date, a reason and a rollback point.
- **Reuse and sharing.** One definition deploys to many regions, environments or teams.
- **Review before apply.** A proposed change can be read and approved before it touches anything.
- **Reduced configuration drift.** A declarative definition keeps deployed reality close to intent.
- **Automation and self-service.** Humans stop being the bottleneck for routine provisioning.

> ON THE EXAM
>
> Versioning, reuse and sharing is the single most repeated correct answer in this domain. Learn that phrase.

What IaC does NOT give you

- **It does not secure your credentials.** Config as code does nothing for secret handling.
- **It does not make resources cheaper.** The same instance costs the same.
- **It does not prevent manual modification.** Console access still changes things.
- **It does not guarantee zero downtime.** Replacing a resource can take it offline.
- **It does not autoscale anything.** Scaling is a property of what you built.

> ON THE EXAM
>
> Every one of these is a real distractor. They sound like benefits because they are benefits of something else.

Mutable or immutable infrastructure?

MUTABLE

Change the thing in place

- Patch and reconfigure live servers
- Each server accumulates its own history
- Drift grows quietly over time
- Debugging asks: what is different here?

IMMUTABLE

Replace the thing entirely

- Build a new version, swap it in
- Every instance is identical by construction
- No accumulated history to diverge
- Upgrades become far less complex

THE TIEBREAKER

The examinable advantage of immutable infrastructure is reduced complexity, not speed and not automation.

Terraform against writing API calls yourself

- **Terraform calls the same provider APIs** you would call. That part is genuinely no different.
- **The difference is the configuration.** Version-controlled, repeatable, and expressed as desired state.
- Terraform tracks **what it already built**, so a second run changes only what differs.
- **It does not replace provider APIs** with its own protocol. Providers are thin wrappers over those APIs.
- **Merely a wrapper, so no real difference** is a distractor. The wrapper is not the point; the state and the graph are.

> ON THE EXAM
>
> Both extremes are wrong. Terraform does not bypass the API, and it is not just a wrapper either.

The workflow this all produces

01

Write

Describe desired infrastructure in configuration files.

02

Plan

Terraform shows exactly what it would create, change or destroy.

03

Apply

The proposed change is executed against the provider APIs.

04

Review

The change lives in version control with an author and a reason.

ON THE EXAM

The plan step is why IaC enables review. It turns an intention into a readable, checkable artefact before anything happens.

Multi-cloud, hybrid cloud, service-agnostic

- **One workflow, many providers.** The same write, plan, apply cycle throughout.
- **Over three thousand providers:** clouds, on-premises, SaaS, DNS, monitoring.
- **Hybrid cloud** means public cloud plus on-premises in one configuration.
- **Service-agnostic** means it only needs a provider that can manage the thing.
- It does **not** make configurations portable. AWS resources stay AWS resources.

> ON THE EXAM
>
> One workflow across providers is the claim. One configuration that runs anywhere is not, and never was.

Terraform or a provider-native tool?

PROVIDER-NATIVE FITS WHEN

One vendor, and staying there

- Everything lives in a single cloud
- You want the vendor to hold state for you
- Deep, same-day support for new services
- CloudFormation, ARM, Deployment Manager

TERRAFORM FITS WHEN

The estate crosses a boundary

- More than one cloud, or cloud plus on-premises
- SaaS and infrastructure in one workflow
- One language and one skill set for all of it
- You accept owning the state file

THE TIEBREAKER

A native tool cannot provision another vendor's platform. That single limitation decides most questions in this pairing.

The trap: secrets and version control

- **Configuration belongs in version control.** Always the right answer.
- **Secrets do not.** Passwords, tokens and keys are never committed alongside it.
- A repository keeps **history**. Deleting a secret later does not remove it.
- Use a **secrets manager**, environment variables, or a run-time fetch.
- Terraform's answer is the **Vault provider**, plus ephemeral values. Module thirteen.

> ON THE EXAM
>
> Two near-identical questions, opposite answers. Configuration in version control is best practice. Secrets in version control never is.

Reading the scenario

THE WORDING SAYS	IT IS POINTING AT
Version, reuse and share configuration	The headline advantage of IaC over a GUI
Deploy into a second cloud or a SaaS platform	A vendor-native tool is the problem, not the answer
Replace rather than patch, identical every time	Immutable infrastructure, and less complex upgrades
Changes should be reviewed before they happen	The plan step, and code review on the repository
Programmatically create and configure resources	The primary purpose of IaC. Not pipelines, not cost
Stop anyone editing resources by hand	IaC cannot promise this. It detects, it does not prevent

ON THE EXAM

This domain is decided by matching a phrase to a definition. Slow down and read the exact claim being made.

> Halden Logistics runs its platform on a single public cloud. Engineers provision resources through the web console. Twice this quarter, an outage was traced to a setting somebody changed by hand and could not account for afterwards.
>
> The engineering director proposes adopting Terraform, and tells the board it will make it impossible for engineers to change infrastructure outside the approved process.

How should the platform lead respond to that claim?

A Agree. Adopting Terraform prevents changes made outside the configuration

B Correct it. Terraform surfaces such changes as drift, but cannot prevent them

C Agree, provided the configuration is stored in version control

D Correct it. Terraform prevents console changes only for immutable resources

◆ PAUSE AND DECIDE

How should the platform lead respond to that claim?

A Terraform prevents it

Overclaims. Terraform holds no authority over the provider's API. Prevention needs provider-side permissions or policy, not a tool choice.

B It surfaces drift, not prevents

Correct. Terraform compares desired state against reality and reports the difference. Detection is real and valuable; prevention is a different control.

C True with version control

Version control adds history and review to the configuration. It has no reach at all over what somebody does in a console.

D Only for immutable resources

Invents a rule. Immutability describes how you choose to change resources, not whether the provider accepts edits from elsewhere.

THE SIGNAL

IaC detects unmanaged change. Preventing it requires taking away the permission to make it.

> Meridian Health standardised on a single cloud vendor's native infrastructure-as-code service two years ago. The templates are mature and the team is comfortable with them.
>
> A new product requires a managed database in a second cloud, and must register its endpoints with an external monitoring SaaS at provisioning time. The team wants one review and approval process covering the whole deployment.

Which aspect of the current approach presents the difficulty?

A Native tools cannot store their templates in version control

B Native tools cannot provision resources outside their own vendor

C Native tools cannot express dependencies between resources

D Native tools cannot be reused across regions

◆ PAUSE AND DECIDE

Which aspect of the current approach presents the difficulty?

A. Cannot use version control

False. A native template is a text file and versions perfectly well. Version control is a practice, not a tool feature.

B. Cannot leave their own vendor

Correct. Two boundaries are crossed here, a second cloud and a SaaS platform, and the native service reaches neither.

C. Cannot express dependencies

False, and it is one of the things native tools do well. Dependency ordering is core to any declarative provisioning tool.

D. Cannot be reused across regions

False. Reuse across regions is a standard, well-supported pattern. The scenario never mentions regions, which is the tell.

THE SIGNAL

The scenario names two boundary crossings and one process requirement. The boundaries decide it; the process requirement explains why it matters.

A team runs forty application servers. Patching is done in place by a configuration management tool. Over eighteen months, six servers have developed behaviour the team cannot reproduce anywhere else, and troubleshooting usually begins by comparing one server against another.

An architect proposes moving to an immutable model: build a new machine image for every release and replace the fleet.

What is the principal advantage the team gains?

A. Upgrades complete more quickly than in-place patching

B. Upgrades become less complex, because instances cannot diverge

C. Upgrades happen automatically without team involvement

D. Upgrades can be applied in place with no downtime

◆ PAUSE AND DECIDE

What is the principal advantage the team gains?

A. Quicker upgrades

Usually the opposite. Building an image and replacing forty machines takes longer than patching them where they stand.

B. Less complex upgrades

Correct. Every instance is built the same way from the same image, so the class of problem described in the scenario stops occurring.

C. Automatic upgrades

Automation is orthogonal. In-place patching here is already automated by a configuration management tool, and it still produced the problem.

D. In-place with no downtime

Describes mutable infrastructure, which is what they are leaving. Replacement can also mean a brief interruption per instance.

THE SIGNAL

Six servers nobody can reproduce is the planted detail. It names divergence, and divergence is exactly what replacement removes.

> A team is moving its Terraform configuration into a shared Git repository so that every engineer can run the same deployment. The configuration needs a database administrator password.
>
> An engineer suggests committing a variables file containing the password, so that anyone who clones the repository can apply immediately without extra setup. She notes the repository is private and access is limited to the eight people on the team.

How should the team proceed?

A. Commit the file. A private repository with eight members is an acceptable boundary

B. Keep the password out of the repository and supply it from a secrets manager or environment

C. Keep the Terraform configuration out of version control entirely, since it references a secret

D. Commit the file, then remove the password in a follow-up commit before granting wider access

◆ PAUSE AND DECIDE

How should the team proceed?

A Private repo is enough

Access scope is not the issue. The secret is now in permanent history, in plaintext, and repository membership changes over time.

B Keep it out entirely

Correct. Configuration is versioned; the secret arrives at run time from a secrets manager, an environment variable, or the Vault provider.

C Don't version the config

Overcorrects, and throws away the entire practice. Referencing a secret is normal; storing its value is the problem.

D Commit then remove later

The most dangerous option, because it feels like a plan. A later commit does not erase history. That password is compromised from the moment it lands.

THE SIGNAL

Two questions that look identical have opposite answers. Configuration in version control, yes. Secrets in version control, never.

Module 1 recap

- **Version, reuse and share.** The headline advantage of code over a graphical interface.
- **Primary purpose:** programmatically create and configure resources. Not pipelines, not cost, not scaling.
- IaC **detects** unmanaged change. It never prevents it.
- **Immutable means less complex,** not quicker. Replacement removes divergence, and often takes longer.
- **One workflow across providers** is the multi-cloud claim. Portable configuration is not.
- **Configuration in version control, always. Secrets, never.**

> ON THE EXAM
>
> Domain 1 is definitional. It is the cheapest domain to score full marks in, and the easiest to lose marks in by reading quickly.